Textora Bulk SMS & API Gateway

Architectural & Technical Specifications

---

1. System Overview

Textora is a high-speed, enterprise-grade Bulk SMS broadcast platform and developer API gateway. It operates as a dual-channel retail SMS management platform:

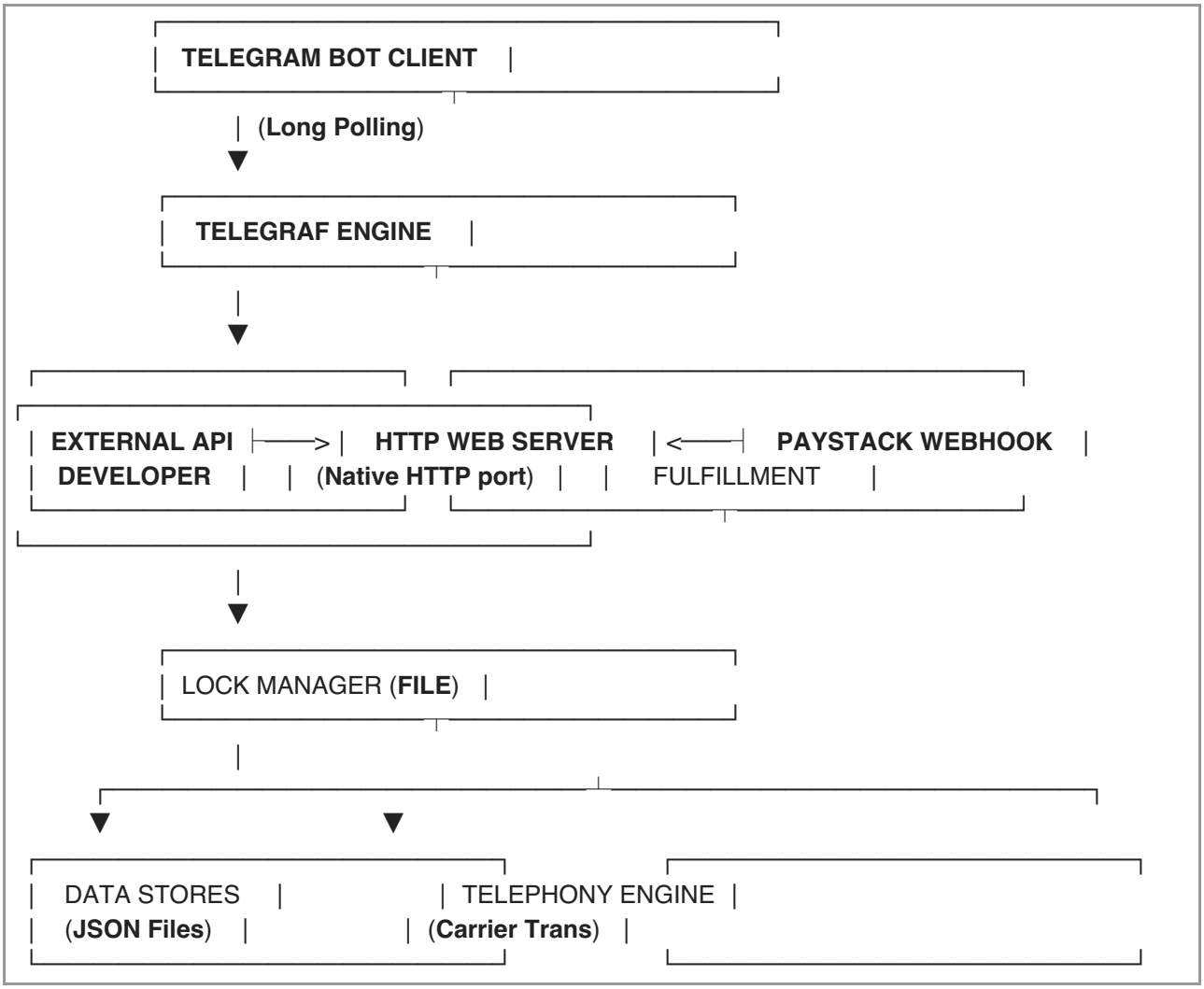
1. **Consumer Portal (Telegram Bot)**: An interface that enables individuals and non-technical business owners to upload contact lists, compose messages, customize sender identities, and execute bulk SMS campaigns on a pay-as-you-go basis.
2. **Developer API (REST Gateway)**: A lightweight, developer-focused API that allows external systems to integrate SMS capabilities (such as One-Time Passwords, alerts, and customer notifications) directly into their custom backend codebases using a prepaid balance model.

Textora includes built-in concurrency protection, local persistent transaction locking, and a self-healing background worker that recovers from server crashes without dropping messages or double-billing clients.

---

2. Architectural Design & Boundaries

Textora is designed using a clean, modular service-oriented architecture written in Node.js. It operates without heavy web frameworks, running directly on Node's native network and filesystem libraries to achieve optimal throughput and low resource utilization.



The system is separated into the following key layers:

2.1. Web Server Layer

Exposed through a single native HTTP listener (port 3000 or environment configuration). It handles incoming webhooks from Paystack (payment gateway) and routing for developer API integrations.

2.2. Messaging Layer

Uses the Telegraf library to handle all bot commands, keyboard configurations, user states, and interactive approval notifications. It communicates with Telegram's servers via long polling.

2.3. Data Storage Layer

File-based database system using transactional JSON files. It avoids external database server overhead, guaranteeing instant data reads/writes while keeping operations local and portable.

2.4. Telephony Integration Layer

Talks directly to the downstream telephony carrier gateway to submit messages, fetch real-time delivery status, and verify carrier balances.

---

3. The Developer Prepaid Balance & Recharge Model

To allow external developers to integrate Textora, the platform implements a prepaid billing model using Paystack as the financial merchant:

1. **API Key Generation**: Developers run the /apikey command in the Telegram bot to register. The bot generates a profile and a unique API Key (txt_...) mapped to their Telegram User ID.
2. **Package Selection**: Developers trigger a recharge (via the bot using /recharge or via the POST /api/v1/recharge endpoint). They select a fixed package plan:
    - **GHS 20** — 400 SMS units (₵0.05 per SMS)
    - **GHS 50** — 1,000 SMS units (₵0.05 per SMS)
    - **GHS 100** — 2,000 SMS units (₵0.05 per SMS)
    - **GHS 200** — 4,000 SMS units (₵0.05 per SMS)
    - **GHS 500** — 10,000 SMS units (₵0.05 per SMS)
    - **GHS 1,000** — 20,000 SMS units (₵0.05 per SMS)
    - **GHS 2,000** — 40,000 SMS units (₵0.05 per SMS)
    - **GHS 5,000** — 100,000 SMS units (₵0.05 per SMS)
3. **Checkout & Payment**: The system initializes a transaction via Paystack and returns a checkout link. The developer completes the payment.
4. **Owner Manual Approval**: Once the payment is verified, the Paystack webhook transitions the payment session to a pending approval state. The bot sends a message to the bot owner's Telegram account (DEVELOPER_CHAT_ID) with inline buttons:
    - [Approve Recharge]
    - [Reject Recharge]
5. **Funding**: The bot owner completes the upstream top-up and clicks **Approve Recharge**. The bot credits the developer's prepaid balance and alerts them.

---

4. API Endpoints (REST Specification)

Developers authenticate by providing their key in the HTTP headers:

- Authorization: Bearer <API_KEY> or x-api-key: <API_KEY>.

4.1. Get Balance

- **URL**: GET /api/v1/balance
- **Description**: Returns the remaining prepaid SMS units balance.
- **Response**:

```
{
  "ok": true,
  "balance": 1450
}
```

4.2. List Verified Sender IDs

- **URL**: GET /api/v1/sender-ids
- **Description**: Returns the list of sender names approved for this account.

- **Query Params**: userId (optional, admin only).
- **Response**:

```
{
  "ok": true,
  "senderIds": ["MYBRAND", "ALERT"]
}
```

4.3. Send SMS

- **URL**: POST /api/v1/send-sms
- **Description**: Sends messages to one or more recipients.
- **Request Body**:

```
{
  "to": ["233201234567", "233207654321"],
  "message": "Verify your login with code: 8876.",
  "senderId": "MYBRAND"
}
```

- **Response**:

```
{
  "ok": true,
  "total_units_debited": 2,
  "remaining_balance": 1448.00,
  "results": [
    { "recipient": "233201234567", "status": "success", "data": { "code": "ok", "message": "Submitted successfully" } },
    { "recipient": "233207654321", "status": "success", "data": { "code": "ok", "message": "Submitted successfully" } }
  ]
}
```

4.4. Fetch API Logs

- **URL**: GET /api/v1/logs
- **Description**: Returns the developer's API SMS transmission logs.
- **Response**:

```
{
 "ok": true,
 "logs": [
  {
   "userId": "12345678",
   "timestamp": "2026-08-14T09:45:00.000Z",
   "message": "Verify your login with code: 8876.",
   "recipientsCount": 2,
   "unitsDebited": 2,
   "remainingBalance": 1448.00,
   "status": "success"
  }
 ]
}
```


4.5. Request Recharge

- **URL**: POST /api/v1/recharge
- **Description**: Creates a Paystack payment session to purchase SMS credits.
- **Request Body**:


```
{
 "planId": 3
}
```


- **Response**:


```
{
 "ok": true,
 "plan_id": 3,
 "amount_ghs": "100.00",
 "credits": 2000,
 "checkout_url": "https://checkout.paystack.com/...",
 "reference": "ref_abc123"
}
```


---

5. Database Schemas (Data Storage Specification)

The local JSON data stores are detailed below:

5.1. api-developers.json

Stores registered developers, mapping their API keys and Telegram user IDs:

```
{
 "api_keys": {
  "txt_abcdef1234567890": {
   "userId": "12345678",
   "apiKey": "txt_abcdef1234567890",
   "balance": 1448.00,
   "name": "Kwadwo Code"
  }
 },
 "users": {
  "12345678": {
   "userId": "12345678",
   "apiKey": "txt_abcdef1234567890",
   "balance": 1448.00,
   "name": "Kwadwo Code"
  }
 }
}
```


5.2. api-usage-log.json

Maintains records of API transmissions:


```
[
 {
  "userId": "12345678",
  "timestamp": "2026-08-14T09:45:00.000Z",
  "message": "Verify your login with code: 8876.",
  "recipientsCount": 2,
  "unitsDebited": 2,
  "remainingBalance": 1448.00,
  "status": "success"
 }
]
```


5.3. payment-sessions.json

Tracks Paystack references and states (regular campaigns & API recharges):


```
{
 "ref_abc123": {
  "price": "100.00",
  "credits": 2000,
  "chatId": 12345678,
  "userId": 12345678,
  "paymentPurpose": "textora_api_recharge",
  "status": "waiting_for_admin_approval",
  "step": "waiting_for_payment"
 }
}
```


---

6. Concurrency Protection & Crash Recovery

6.1. Local File Transaction Locking (lockManager.js)

To avoid race conditions when multiple API calls or webhooks hit the server at the same millisecond, Textora uses a locking mechanism:

- Before reading/writing any shared JSON store (e.g. api-developers.json or payment-sessions.json), the thread must acquire a unique file lock (locks/<lockName>.lock).
- If a lock file exists, other threads wait, checking at a regular interval (retryIntervalMs = 50ms) until it is released or expires (lockExpiryMs = 60s).
- If a lock is found to be older than the expiry window (stale due to a crashed thread), it is deleted and acquired by the new requester automatically.

6.2. Crash Recovery Worker (recoveryWorker.js)

When bulk SMS campaigns are broadcasted, they might consist of tens of thousands of contacts. If the server crashes mid-broadcast:

- The system writes individual progress states into sms-log.json for each contact.
- When the server boots up, it starts the **Textora Recovery Worker**.
- The worker scans payment-sessions.json for any transactions marked as processing that have been active for more than 120 seconds.
- It iterates over the campaign contacts, checks sms-log.json to see which recipients have already received the message, and **continues the broadcast for the remaining contacts** without duplicating any messages.